

Design Decisions & Discrepancies

What the Project Document Specifies vs. What the Code Implements

For each gap: the planned design, the actual implementation, and the engineering reason why.

Summary of Discrepancies

The table below maps each gap between the project specification and the implemented codebase:

#	Topic	What Document Says	What Code Does
1	Model Architecture	SVM + Bi-LSTM with Attention	Random Forest (500 trees)
2	Signal Filtering	Band-pass 1–45 Hz + Notch 50 Hz on raw EEG	Not applied — Muse CSV has pre-computed powers
3	PSD / Feature Extraction	Welch's method on raw microvolt signal	Direct averaging of Muse band power columns
4	Hjorth Parameters	Activity, Mobility, Complexity from raw signal	Not implemented — no raw EEG time-series used
5	HRV / PPG Features	RMSSD from PPG for heart rate variability	Not implemented — PPG column not processed

1. Machine Learning Model

Document specifies Two-tiered approach: Tier 1 — SVM with RBF kernel as baseline classifier Tier 2 — Bidirectional LSTM (Bi-LSTM) with Attention to capture temporal sequences across windows	Code implements Single model only: RandomForestClassifier (500 trees, Gini criterion) No SVM, no LSTM, no attention mechanism anywhere in either train.py or test.py
--	---

Engineering Reason The SVM and Bi-LSTM were the intended architecture described in the design phase.
--

The Random Forest was chosen as a practical substitute during implementation for the following concrete reasons:

1. **Data size.** The benchmark dataset has ~342 rows — one per 5-minute session window. Bi-LSTMs need hundreds of labelled sequential samples per class to learn meaningful temporal patterns. With this sample count, an LSTM would overfit severely.
2. **Feature type.** The input is already aggregated band-power features, not a raw time-series. An LSTM is designed to learn from raw sequences; feeding it pre-summarised features removes most of the temporal signal it would exploit.
3. **SVM vs. Random Forest.** Both are viable for tabular spectral features. Random Forest was preferred because it: (a) handles class imbalance natively via `class_weight=balanced`, (b) outputs calibrated probabilities via `predict_proba` without a separate Platt scaler, (c) is less sensitive to feature scaling errors during testing.
4. **Interpretability.** Feature importance scores from Random Forest directly show which bands (e.g. theta/alpha ratio) drive classification — useful for validating that the model is learning neurologically meaningful patterns.

To implement the original specification

SVM: Replace `RandomForestClassifier` with `SVC(kernel='rbf', probability=True, class_weight='balanced')`.

Add `GridSearchCV` over `C` and `gamma` parameters.

Bi-LSTM: Requires raw per-sample features (not window-averaged). Each input would be a sequence of shape `(time_steps, n_features)`. Needs significantly more labelled data. Use PyTorch or TensorFlow with a bidirectional LSTM layer + additive attention head.

2. Signal Preprocessing — Band-Pass and Notch Filters

Document specifies

Raw EEG signal is preprocessed with:

Band-pass filter: 1–45 Hz (removes DC drift and

high-frequency noise)

Notch filter: 50 Hz (removes power-line interference from mains electricity)

Applied to continuous microvolt time-series before

Code implements

No filters are applied in `test.py`.

The script reads the Muse CSV and directly uses the

pre-computed band power columns:

`Delta_TP9`, `Theta_AF7`, `Alpha_AF8`, `Beta_TP10`, ...

These are already processed values (0–1 relative power), not raw microvolt waveforms.

any spectral analysis.

Engineering Reason

Band-pass and notch filters are operations on raw, continuous voltage waveforms sampled at 256 Hz. They work by convolving the signal with a filter kernel in the time domain, or equivalently multiplying by a transfer function in the frequency domain.

The Muse 2 CSV does not export that raw waveform in the band power columns. Instead, the Muse hardware runs its own internal DSP pipeline (which includes equivalent filtering) and outputs the result as relative band powers. By the time the data reaches the CSV, the raw waveform no longer exists in those columns — only the summarised power values do.

Therefore, applying a band-pass filter to Delta_TP9 or Alpha_AF7 is mathematically meaningless: you would be filtering a scalar value that represents a power ratio, not a time-varying voltage signal. The correct action is to use the values as-is.

The RAW_TP9 / RAW_AF7 / RAW_AF8 / RAW_TP10 columns in the Muse CSV do contain raw microvolts at 256 Hz. Filtering is valid on those columns.

To apply filtering as specified

Use the RAW_TP9, RAW_AF7, RAW_AF8, RAW_TP10 columns instead of the band power columns.

Apply `scipy.signal.butter + sosfiltfilt` for band-pass (1–45 Hz).

Apply `scipy.signal.iirnotch` for the 50 Hz notch filter.

Then compute Welch PSD on the filtered signal to extract band powers manually.

This gives full control over the DSP chain but requires discarding the Muse's pre-computed values.

3. Feature Extraction — Welch's Method

Document specifies Power Spectral Density (PSD) is estimated using Welch's method on the filtered raw EEG signal. Band powers are extracted by integrating the PSD over each frequency band: delta 1–4 Hz, theta 4–8 Hz, alpha 8–13 Hz,	Code implements test.py header explicitly states: 'This avoids re-running Welch PSD since the Muse already provides band powers.' The script takes <code>mean(Band_TP9, Band_AF7, Band_AF8, Band_TP10)</code> within each 2-second window. No FFT or PSD calculation is performed.
---	---

beta 13–30 Hz, gamma 30–45 Hz
Normalised relative power = band / total power.

Engineering Reason

Welch's method divides a raw signal into overlapping segments, applies a window function (e.g. Hann), computes the FFT of each segment, and averages the squared magnitudes. This produces a smooth estimate of power as a function of frequency (units: $\mu V^2/Hz$). Band power is then obtained by integrating (trapz) the PSD over the band's frequency range.

This whole pipeline requires a continuous, uniformly sampled microvolt time-series. The Muse 2 already performs this computation internally for its 4 channels and outputs the result directly — the Delta_TP9 column IS the relative delta band power for TP9.

Re-running Welch on those output values would be computing 'the spectrum of a spectrum', which is neither equivalent to the original computation nor scientifically meaningful.

The Muse's internal computation is equivalent to the specified Welch pipeline. The code correctly skips the redundant step.

To use Welch's method directly (as specified)

Load RAW_TP9/AF7/AF8/TP10 columns (raw microvolts at 256 Hz).
After band-pass and notch filtering (see Section 2), segment into 2-second windows.
For each window, call `scipy.signal.welch(signal, fs=256, nperseg=256, noverlap=128)`.
Integrate PSD over each band using `numpy.trapz`.
Divide by total power to obtain relative band powers.
This reproduces the Muse's internal computation manually and gives identical feature values.

4. Hjorth Parameters

Document specifies

Temporal / Complexity Features from raw EEG:
Activity — variance of the signal (power)
Mobility — ratio of std of first derivative
 to std of signal
Complexity — ratio of mobility of first derivative
 to mobility of signal
These capture temporal dynamics not described
by spectral analysis alone.

Code implements

Not implemented in either `train.py` or `test.py`.
No Hjorth parameter calculation exists anywhere
in the codebase.
All 19 features are spectral (band powers, spatial
std, ratios, regional aggregates).

Engineering Reason

Hjorth parameters are computed from the raw time-domain EEG signal and its derivatives:

Activity = $\text{var}(x)$ — measures signal power
 Mobility = $\text{std}(dx/dt) / \text{std}(x)$ — measures mean frequency
 Complexity = $\text{Mobility}(dx/dt) / \text{Mobility}(x)$ — measures signal complexity

All three require access to the raw continuous microvolt waveform $x(t)$ sampled at 256 Hz.

The Muse CSV band power columns (Delta_TP9 etc.) do not contain this waveform. The columns contain a single scalar per sample period, representing an already-summarised power value. You cannot compute a meaningful derivative (dx/dt) from these values because they do not represent a continuous voltage — they represent a power estimate that updates approximately once per second.

Therefore Hjorth parameters are not computable from the band power CSV format. They can only be computed from the RAW_TP9/AF7/AF8/TP10 columns.

To add Hjorth parameters

Use RAW_TP9, RAW_AF7, RAW_AF8, RAW_TP10 columns (raw μV waveform).

For each 2-second window of raw signal x :

```
dx = numpy.diff(x)      # first derivative
ddx = numpy.diff(dx)    # second derivative
activity = numpy.var(x)
mobility = numpy.std(dx) / numpy.std(x)
complexity = (numpy.std(ddx)/numpy.std(dx)) / mobility
```

Add these 3 values per channel (4 channels = up to 12 additional features) to the feature vector. Recompute on benchmark data using the available RAW columns if present, or request a raw dataset.

5. HRV / PPG Features (Heart Rate Variability)

Document specifies

Multimodal feature: Heart Rate Variability
 Metric: RMSSD (Root Mean Square of Successive Differences between R-R intervals)
 Source: PPG (photoplethysmography) sensor

Code implements

Not implemented in either train.py or test.py.
 The Muse CSV does contain PPG columns:
 PPG_Ambient, PPG_IR, PPG_Red
 Heart_Rate

Purpose: Holistic view of physiological state — HRV drops under stress/distraction and rises during calm meditative states.

But none of these are read or processed anywhere in the feature extraction pipeline.

Engineering Reason

HRV / RMSSD computation requires detecting individual heartbeat peaks (R-peaks) in the PPG waveform and measuring the intervals between them (R-R intervals, also called NN intervals).

$RR = [t_2 - t_1, t_3 - t_2, t_4 - t_3, \dots]$ (successive R-R intervals in ms)

$RMSSD = \sqrt{\text{mean}(\text{diff}(RR)^2)}$ (Root Mean Square of Successive Differences)

This requires a peak-detection algorithm on the PPG_IR or PPG_Red waveform (e.g. using `scipy.signal.find_peaks` or a dedicated library like `heartpy`).

The benchmark training dataset (Thai monk EEG) does not contain PPG data — it is a pure EEG dataset. This means RMSSD cannot be computed for the training samples, so the model cannot learn a weight for it. Adding a feature that exists only at test time but not in training is a data leakage / feature mismatch problem that would cause the model to fail or be retrained.

The `Heart_Rate` column in the Muse CSV is a pre-computed scalar (beats per minute), not the raw R-R interval sequence needed for RMSSD. It updates too slowly (approximately once every few seconds) to be useful as a per-window HRV feature.

To add HRV / RMSSD as a feature

Requirement: the training dataset must also contain PPG or HR data, or you must re-label and train on a dataset that includes it (e.g. your own recorded sessions).

If using the Muse CSV PPG_IR column:

1. Apply a band-pass filter (0.5–4 Hz) to isolate the cardiac waveform.
2. Detect peaks with `scipy.signal.find_peaks`.
3. Compute R-R intervals: $RR = \text{numpy.diff}(\text{peak_times_in_ms})$.
4. $RMSSD = \sqrt{\text{mean}(\text{numpy.diff}(RR)^2)}$ per 2-second window.
5. Add RMSSD as feature index 20 in the feature vector.
6. Retrain the model on data that includes this feature.

Summary

The four gaps listed above share a common root cause: the benchmark training data (Thai monk dataset) is a pure EEG spectral power dataset, and the Muse 2 test data arrives pre-processed by the headset's own firmware. Both constraints push the implementation away from the raw-signal pipeline described in the design document and toward working directly with the summarised band power values.

The implemented pipeline is internally consistent and scientifically valid for the data it has. The 19 spectral features are well-established EEG meditation markers. The Random Forest is appropriate for the dataset size.

To fully implement the original specification, the following source data changes would be needed:

- Raw μ V EEG time-series (for filtering, Welch PSD, and Hjorth parameters) — use RAW_TP9/AF7/AF8/TP10 columns from Muse CSV
- A training dataset that includes both EEG and PPG/HRV — to allow the model to learn HRV weights
- Sufficient sequential labelled samples per subject — to train a Bi-LSTM meaningfully